**CS 335**
Web Development
Spring 2025/2026

---

**Milestone 3**
Database & Front-End Prototype


**TEAM Members**

Abdulaziz, ID: 2212173741
Yousef, ID: 22221110617
Abdullah, ID: 2231122390


**Project Title:**
Tadreeb Web Application

## Introduction

The front-end of Tadreeb is implemented as a React single-page application that provides the user interface for students, companies, and administrators. It is responsible for presenting public information, handling authentication screens, displaying role-based dashboards, and connecting users to the platform features through reusable pages and components.

The application follows a modular structure so that each role has its own pages while shared elements, such as navigation, portal layout, authentication guards, cards, and modals, are kept in reusable component folders. This organization helps keep the interface consistent, easier to maintain, and ready to communicate with the backend API services.

# 1 Front-End React Project

## 1.1 Folder Structure

The Tadreeb front-end project is organized using a modular React structure to ensure clarity and scalability. The main application code is located inside the `src` folder, and the page files are grouped based on user roles as follows:

- **Public:** Home
- **Authentication:** Login, Signup
- **Student:** Home, Profile, Applications
- **Company:** Company Request Form, Dashboard, Programs, Participants, Profile
- **Admin:** Dashboard, Programs, Participants, Users, Companies, Profile

**Components:** Reusable UI elements are organized within the `components` folder. This includes common components such as `Navbar` and `Footer`, authentication route wrappers such as `ProtectedRoute` and `PublicOnlyRoute`, portal layout components such as `PortalLayout`, `PortalSidebar`, `PortalTopbar`, `PortalLoader`, and `PortalStatCard`, and role-specific components for students, companies, and administrators.

**Assets and Styles:** The `assets` folder stores static resources such as hero images, login/signup images, chart images, company/program images, and partner logos. The `styles` folder contains CSS files grouped by feature area, including home, authentication, student, company, admin, portal, layout, and shared variable styles.

**API, Context, and Utilities:** The `api` folder contains service files for authentication, admin operations, applications, company requests, and opportunities. The `context` folder contains `AuthContext.js`, which manages authentication state across the application. The `utils` folder includes helper files for tokens, password rules, link formatting, company account status, and program status.

**Application Entry Points:** The files `App.js` and `index.js` are responsible for rendering the application and managing routing between different pages. This project does not use `main.jsx`; `index.js` is the React entry point.

This structure reflects the current front-end implementation and its connection points with the backend API, as illustrated in Figure 1.

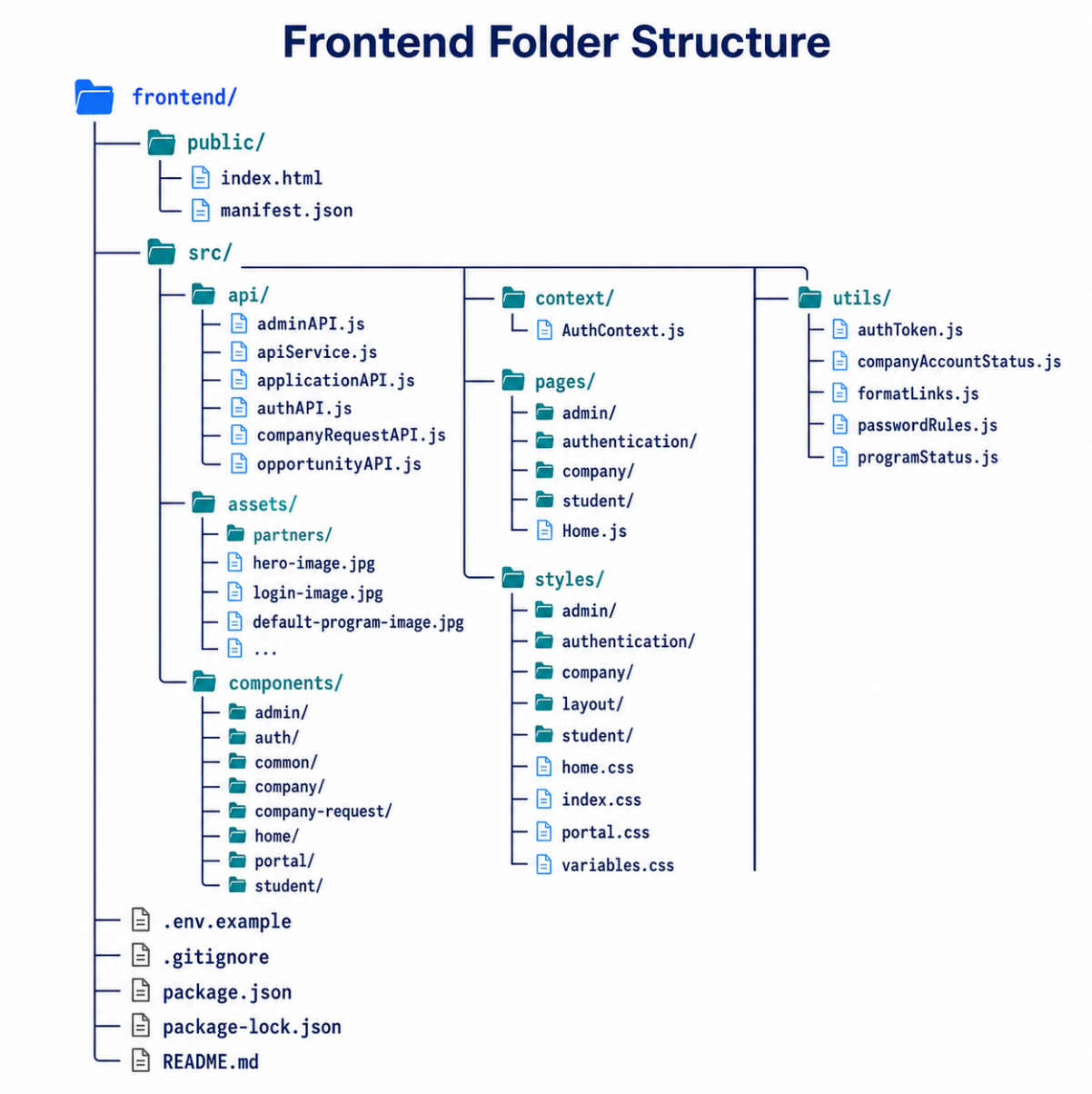


Figure 1: Frontend Folder Structure

## 1.2 UI Components

The Tadreeb platform uses a modular, component-based front-end structure. The interface is organized according to user roles: Public, Student, Company, and Admin. Each page is built from reusable components such as layout wrappers, navigation bars, cards, search tools, and modals. This improves consistency, maintainability, and scalability across the platform.

### Component Hierarchy

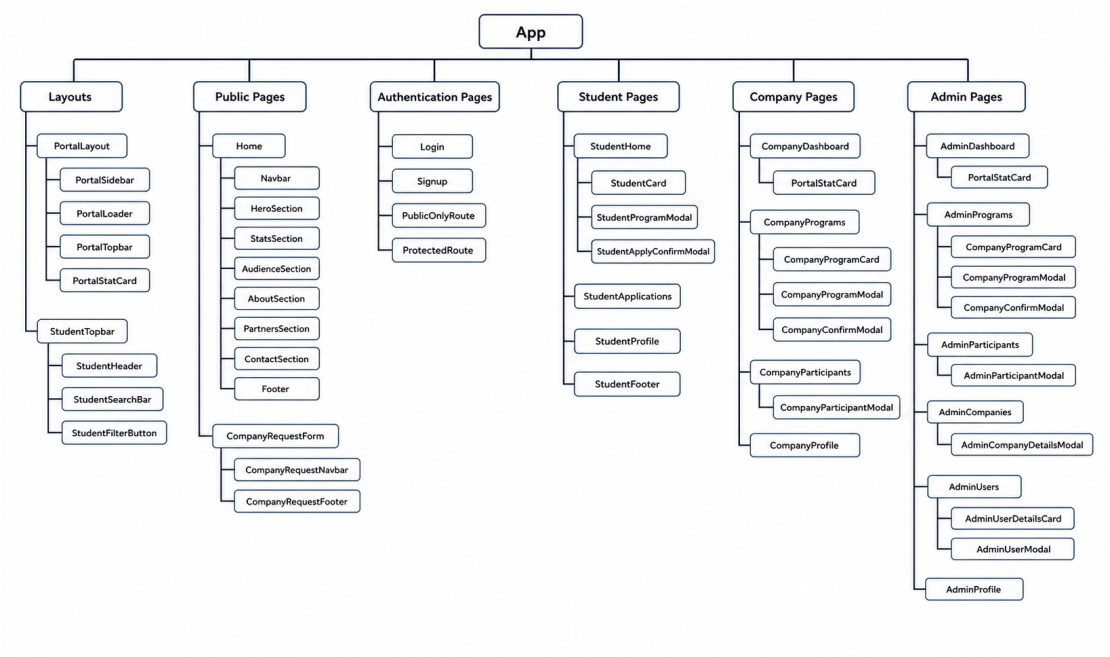


Figure 2: UI Component Hierarchy of the Tadreeb Platform

### Description of UI Components

- **Navbar and Footer:** Shared public navigation components used by the home page.
- **ProtectedRoute and PublicOnlyRoute:** Authentication wrapper components used by `App.js` to control access to protected pages and public-only pages.
- **Portal Components:** Shared dashboard components used across the student, company, and admin portals. These include layout, sidebar, topbar, loading, and statistics card components.
- **Home Components:** Components used to build the public home page, including the hero, about, audience, statistics, partners, and contact sections.
- **Student Components:** Components used in the student interface, including the topbar, page header, search bar, filter button, program card, program details modal, application confirmation modal, and footer.

- **Company Components:** Components used in the company dashboard to support program management, program details, participant details, and confirmation dialogs.
- **Company Request Components:** Components used by the company join request page, including the request page navbar and footer.
- **Admin Components:** Components used in administrator pages to manage users, participants, company requests, and detailed information modals.

Overall, this structure shows the parent–child relationship used in the front end. `App.js` controls routing, pages are grouped by role, and each page imports smaller reusable components from the `components` folder.

## 1.3 Integration of Mock Data

Since the Tadreeb platform is currently in the prototype stage and not yet connected to a live MongoDB database, a modular mock data approach is used to simulate real-world system behavior. Static JavaScript files are stored inside the `src/data/` directory, each corresponding to one of the five database collections: `students.js`, `companies.js`, `programs.js`, `applications.js`, and `admins.js`.

These files are imported directly into the relevant page components and rendered dynamically using the `.map()` function, allowing the UI to display realistic data without any backend connection. Below is the mock data defined for each collection.

**students.js**

```
// src/data/students.js
const students = [
  {
    studentID: 1,
    universityID: "KU2023001",
    firstName: "Ahmad",
    lastName: "Ali",
    email: "ahmad.ali@mail.com",
    password: "hashed123",
    mobileNo: "96590000001",
    gender: "Male",
    universityName: "Kuwait University",
    major: "CS",
    year: "Third",
    skills: ["Python", "SQL"],
  },
  {
    studentID: 2,
    universityID: "KU2023002",
    firstName: "Sara",
    lastName: "Hassan",
    email: "sara.h@mail.com",
    password: "hashed456",
    mobileNo: "96590000002",
    gender: "Female",
    universityName: "Kuwait University",
    major: "IT",
    year: "Second",
    skills: ["Java", "Teamwork"],
  },
  {
    studentID: 3,
    universityID: "AUK2023003",
    firstName: "John",
    lastName: "Smith",
    email: "john.s@mail.com",
    password: "hashed789",
    mobileNo: "96590000003",
    gender: "Male",
    universityName: "AUK",
    major: "Engineering",
    year: "Fourth",
    skills: ["C++", "Leadership"],
  },
];
export default students;
```

**companies.js**

```
// src/data/companies.js
const companies = [
  {
    companyID: 1,
    companyName: "TechSoft",
    email: "info@techsoft.com",
    industry: "IT",
    phone: "+96590011111",
    website: "www.techsoft.com",
    size: "50-100",
    location: "Kuwait",
    foundedYear: 2015,
    contactPerson: "Khalid Ahmad",
    description: "Software solutions company.",
    joinReason: "Find interns",
    status: "Active",
  },
  {
    companyID: 2,
    companyName: "BuildCorp",
    email: "hr@buildcorp.com",
    industry: "Engineering",
    phone: "+96590022222",
    website: "www.buildcorp.com",
    size: "100-200",
    location: "UAE",
    foundedYear: 2010,
    contactPerson: "Sara Lee",
    description: "Construction firm.",
    joinReason: "Recruit talent",
    status: "Pending",
  },
];

export default companies;
```

**programs.js**

```
// src/data/programs.js
const programs = [
  {
    programID: 1,
    title: "Web Dev Internship",
    subtitle: "Learn Web",
    description: "Web development training.",
    rules: "Attendance required.",
    qualifications: "Basic HTML, CSS, and JavaScript.",
    location: "Remote",
    seats: 10,
    dateFrom: "2026-06-01",
    dateTo: "2026-08-01",
    registrationDeadline: "2026-05-20",
    imageURL: "img1.com",
    companyID: 1,
    status: "Active",
  },
  {
    programID: 2,
    title: "AI Training",
    subtitle: "Intro to AI",
    description: "AI fundamentals training.",
    rules: "Python required.",
    qualifications: "Python basics.",
    location: "Office",
    seats: 5,
    dateFrom: "2026-07-01",
    dateTo: "2026-09-01",
    registrationDeadline: "2026-06-20",
    imageURL: "img2.com",
    companyID: 2,
    status: "Active",
  },
];

export default programs;
```

**applications.js**

```
// src/data/applications.js
const applications = [
  {
    appID: 1,
    studentID: 1,
    programID: 1,
    status: "Submitted",
    appliedDate: "2026-03-01",
    decisionNote: "-",
    adminStatus: "Review",
    adminNote: "",
    visibleToStudent: true,
  },
  {
    appID: 2,
    studentID: 2,
    programID: 1,
    status: "Accepted",
    appliedDate: "2026-03-02",
    decisionNote: "Strong skills",
    adminStatus: "Accepted",
    adminNote: "Application reviewed.",
    visibleToStudent: true,
  },
  {
    appID: 3,
    studentID: 3,
    programID: 2,
    status: "Rejected",
    appliedDate: "2026-03-03",
    decisionNote: "Not qualified",
    adminStatus: "Rejected",
    adminNote: "Application reviewed.",
    visibleToStudent: true,
  },
];

export default applications;
```

**admins.js**

```
// src/data/admins.js
const admins = [
  {
    adminID: 1,
    firstName: "Ali",
    lastName: "Hassan",
    email: "admin1@mail.com",
    phone: "+96591111111",
    password: "hashadmin1",
    jobTitle: "System Administrator",
    status: "Active",
    role: "admin",
  },
  {
    adminID: 2,
    firstName: "Noor",
    lastName: "Saleh",
    email: "admin2@mail.com",
    phone: "+96592222222",
    password: "hashadmin2",
    jobTitle: "Platform Manager",
    status: "Active",
    role: "admin",
  },
];

export default admins;
```

# II. Database Design

## Introduction

The Tadreeb database is engineered as a document-oriented NoSQL system built on **MongoDB**, designed to serve as the structural backbone of a centralized internship marketplace. Its primary function is to manage the data lifecycle of three distinct user roles: Students, Companies, and Administrators.

From a data perspective, the system is built to facilitate:

- **Identity and Access Management:** Utilizing role-based authentication collections to secure unique credentials and profile data for each user type.
- **Marketplace Logic:** Storing and retrieving dynamic internship listings while supporting multi-parameter filtering for location, duration, and compensation.
- **Workflow Persistence:** Tracking the state transitions of student applications from submission through to final review, ensuring data consistency across the student and company dashboards.

Relationships between collections are managed through referenced **ObjectIDs**, ensuring that all interactions — from account approvals to application tracking — are processed accurately within a scalable and flexible environment.

## Database Schema

### Collection: Student

| Field | Type / Rule | Description |
|---|---|---|
| `_id` | ObjectId, PK | MongoDB primary identifier. |
| `universityID` | String, unique | Student university identification number. |
| `firstName`, `lastName` | String | Student first and last name. |
| `email` | String, unique | Login email, stored in lowercase and validated as an email. |
| `password` | String | Hashed password, excluded from normal queries. |
| `mobileNo` | String | Student contact phone number. |
| `gender` | Enum | Allowed values: Male or Female. |
| `universityName`, `major` | String | Academic institution and major. |
| `year` | Enum | Allowed values: First, Second, Third, Fourth, or Fifth. |
| `skills` | Array of String | Student technical and soft skills. |
| `role`, `status` | String / Enum | Role is student; status can be active, inactive, or pending. |
| `createdAt`, `updatedAt` | Date | Automatic timestamps. |

**Collection: Company**

| Field | Type / Rule | Description |
|---|---|---|
| `_id` | ObjectId, PK | MongoDB primary identifier. |
| `companyName` | String | Registered company name. |
| `email` | String, unique | Company login email, stored in lowercase and validated as an email. |
| `password` | String | Hashed password, excluded from normal queries. |
| `industry`, `phone`, `location` | String | Company sector, phone number, and location. |
| `website`, `size` | String | Optional website and company size values. |
| `foundedYear` | Number | Company founding year, validated against allowed year range. |
| `contactPerson` | String | Main contact person for the company account. |
| `description`, `joinReason` | String | Company overview and reason for joining Tadreeb. |
| `status`, `role` | Enum / String | Status can be Pending, Active, Inactive, or Rejected; role is company. |
| `createdAt`, `updatedAt` | Date | Automatic timestamps. |

**Collection: Admin**

| Field | Type / Rule | Description |
|---|---|---|
| `_id` | ObjectId, PK | MongoDB primary identifier. |
| `firstName`, `lastName` | String | Admin first and last name. |
| `email` | String, unique | Admin login email, stored in lowercase. |
| `phone`, `jobTitle` | String | Contact number and job title. |
| `password` | String | Hashed password, excluded from normal queries. |
| `gender` | Enum | Allowed values: Male, Female, or empty. |
| `country`, `language`, `extraInfo` | String | Optional profile details. |
| `status`, `role` | String | Status is Active by default; role is admin. |
| `createdAt`, `updatedAt` | Date | Automatic timestamps. |

**Collection: CompanyRequest**

| Field | Type / Rule | Description |
|---|---|---|
| `_id` | ObjectId, PK | MongoDB primary identifier. |
| `companyName`, `industry` | String | Company name and sector from the join request form. |
| `officialEmail` | String | Official email address, stored in lowercase and validated. |
| `phoneNumber`, `website`, `location` | String | Contact and location details. |
| `companySize`, `foundedYear` | String | Company size and founding year submitted in the request. |
| `contactPerson` | String | Person responsible for the request. |
| `companyDescription`, `joinReason` | String | Request description and reason for joining Tadreeb. |
| `confirmInfo` | Boolean | Confirms that submitted information is accurate. |
| `status` | Enum | Request status: Pending, Approved, or Rejected. |
| `createdAt`, `updatedAt` | Date | Automatic timestamps. |

**Collection: Opportunity**

| Field | Type / Rule | Description |
|---|---|---|
| `_id` | ObjectId, PK | MongoDB primary identifier. |
| `title`, `subtitle` | String | Opportunity title and short subtitle. |
| `description`, `rules`, `qualifications` | String | Opportunity details, rules, and required qualifications. |
| `location` | String | Opportunity location. |
| `seats` | Number | Number of seats, limited between 1 and 20. |
| `dateFrom`, `dateTo` | Date | Start and end dates; end date must be after start date. |
| `registrationDeadline` | Date | Deadline to apply; must be before the start date. |
| `imageURL` | String | Image used for the opportunity card. |
| `companyID` | ObjectId, FK | References the company that created the opportunity. |
| `status` | Enum | Opportunity status: Active or Completed. |
| `createdAt`, `updatedAt` | Date | Automatic timestamps. |

**Collection: Application**

| Field | Type / Rule | Description |
|---|---|---|
| `_id` | ObjectId, PK | MongoDB primary identifier. |
| `studentID` | ObjectId, FK | References the student who submitted the application. |
| `programID` | ObjectId, FK | References the opportunity/program being applied for. |
| `status` | Enum | Student-facing application status: Submitted, Under Review, Not Reviewed, Accepted, or Rejected. |
| `appliedDate` | Date | Date when the application was submitted. |
| `decisionNote` | String | Company feedback or decision note. |
| `adminStatus`, `adminNote` | Enum / String | Admin review status and optional admin note. |
| `visibleToStudent` | Boolean | Controls whether the application is visible to the student. |
| `createdAt`, `updatedAt` | Date | Automatic timestamps. |

**Relationship Summary:** To explain the operational logic of the **Tadreeb** platform, the following relational connections are established within the database:

- **One-to-Many (Company to Program):** A single verified company entity is capable of publishing and managing multiple internship and training opportunities.
- **One-to-Many (Student to Application):** A registered and active student is permitted to apply for multiple different programs to maximize their training prospects.
- **One-to-Many (Program to Application):** Each individual internship listing is designed to receive and organize applications from numerous unique students for company review.

## Sample Data

Table 1: Sample Data for Student

| studentID | Name | Email | Major / Year | Skills |
|---|---|---|---|---|
| 1 | Ahmad Ali | ahmad.ali@mail.com | CS / Third | Python, SQL |
| 2 | Sara Hassan | sara.h@mail.com | IT / Second | Java, Teamwork |
| 3 | John Smith | john.s@mail.com | Engineering / Fourth | C++, Leadership |

Table 2: Sample Data for Company

| companyID | Company Name | Email | Industry | Status |
|---|---|---|---|---|
| 1 | TechSoft | info@techsoft.com | IT | Active |
| 2 | BuildCorp | hr@buildcorp.com | Engineering | Pending |

Table 3: Sample Data for Program

| programID | Title | Location | Dates | Seats |
|---|---|---|---|---|
| 1 | Web Dev Internship | Remote | 2026-06-01 to 2026-08-01 | 10 |
| 2 | AI Training | Office | 2026-07-01 to 2026-09-01 | 5 |

Table 4: Sample Data for Application

| appID | studentID | programID | Status | Decision Note |
|---|---|---|---|---|
| 1 | 1 | 1 | Submitted | - |
| 2 | 2 | 1 | Accepted | Strong skills |
| 3 | 3 | 2 | Rejected | Not qualified |

Table 5: Sample Data for SystemAdmin

| adminID | Name | Email | Phone | Role |
|---|---|---|---|---|
| 1 | Ali Hassan | admin1@mail.com | +96591111111 | Admin |
| 2 | Noor Saleh | admin2@mail.com | +96592222222 | Admin |

# III. Contributions

| Team member | Contribution | |
|---|---|---|
| | Who did what | Percentage (%) |
| Abdulaziz | Responsible for Part 1 of Task 1.1 (Folder Structure) and contributed to Task 2, including defining parts of the Database Schema and Data Types. | 40% |
| Yousef | Mock Data 1.3 and Database Design - Sample Data. | 40% |
| Abdullah | UI Components 1.2 - based on child/parent relations | 20% |